

• BUILD JOURNEY • LEVEL 8 COMPLETE

Speech Translator

Real-time captions for online meetings — designed, benchmarked, and built with LLMs as an active collaborator, not an autocomplete.

14

SESSIONS

72

DECISIONS

5.9s

P95 WORD AGE

133

TESTS PASSING

4 GB

VRAM BUDGET



THE ASSIGNMENT

Starting with a brief — and a missing line

The assignment was to build a real-time speech translator with a UI, minimal latency, translated text output, and bonus marks for silence detection and punctuation. There was one wrinkle — three lines of the brief were truncated in the copy received:

*"It should be able to translate speech between **[MISSING — TRUNCATED]**"*

Rather than assume, the docs recorded this as a blocker. The design proceeded on the safest interpretation: **incoming speech only**, one direction, auto-detected source language. If the missing text said "between two people," the input layer is pluggable and microphone capture can be added as a small delta.

Two requirements were unambiguous:

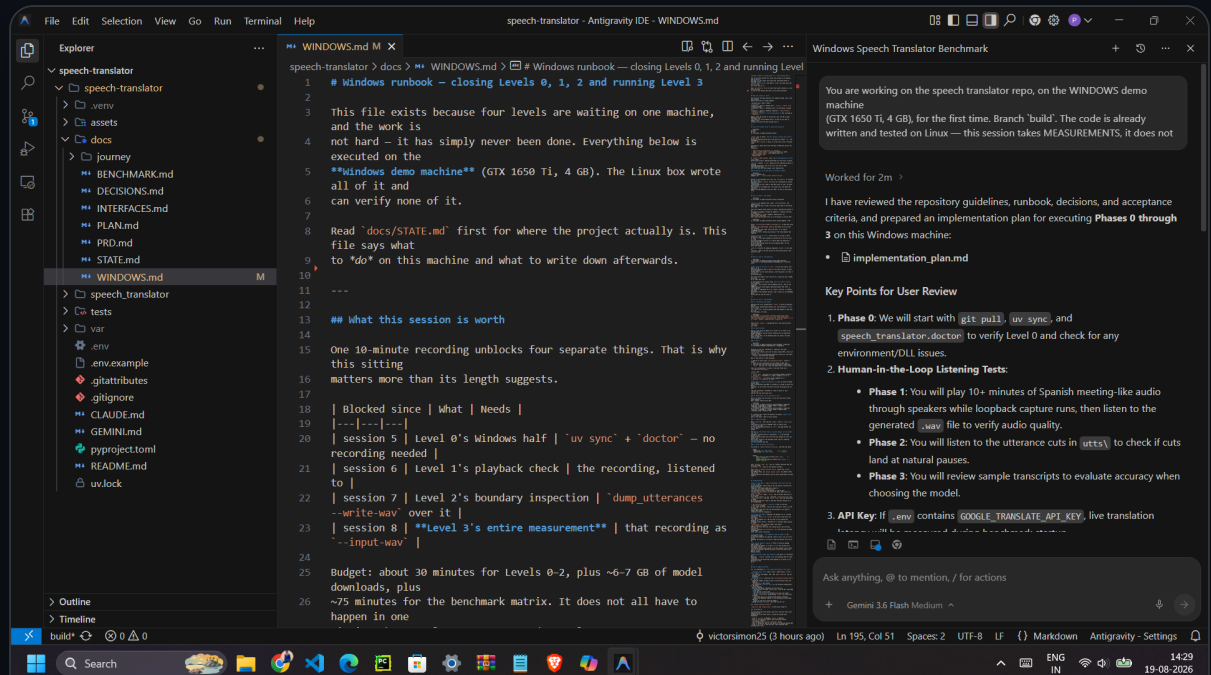
- **Document the LLM journey** — prompts, screenshots, LLM assessments, recovery from failures
- **Use LLMs extensively** — from initial idea through coding, testing, and evaluation

"This is not a test to check your existing knowledge and capabilities. We want to see how you use LLMs to overcome the limitations of your knowledge."

That framing shaped every session that followed: the LLM was a senior engineer to be challenged, not a code generator to be accepted blindly.

Architecture — designed before a line of code was written

Session 0 was a **chat-only design session** with Claude (Opus 5) in Claude Code. The explicit instruction to the model: *"do not agree with me, point out where I'm wrong, and don't state anything you aren't sure of or don't have real numbers for."* That instruction changed the session's output materially.



- Antigravity IDE — 'WINDOWS.md' open alongside the Gemini planning panel laying out Phases 0–3 for the Windows demo machine. The first time the LLM saw the target hardware: GTX 1650 Ti, 4 GB VRAM.

What the LLM got right

- **Refused to guess at hardware.** Instead of estimating Whisper RTF, it ran `nvidia-smi` — finding no NVIDIA GPU on the dev box. That single fact drove the platform pivot to Windows.
- **Verified the audio path rather than asserting it.** Claimed PipeWire exposes a monitor source, then confirmed it via `pw-cli dump`. Same discipline for Windows: found PyAudioWPatch with the actual wheel support matrix checked.
- **Corrected two wrong assumptions.** Speaker tags from loopback audio are architecturally impossible — the meeting client discards identity before the OS sees audio. "Two-way translation" and "speaker tagging" are different axes; the model separated them cleanly.
- **Found the hard stop.** Teams' real-time media platform is C#/.NET only, requires Windows Server — a binary blocker for a Python project on a deadline.

Where the model needed pushing

- Repeated the same blockers across five turns. Correct — they genuinely gate scope — but the *why* had to be drawn out explicitly by the user.
- On RTF, was slow to explain the key insight: the question is not "how fast is Whisper" but whether it runs faster than audio arrives. That binary distinction had to be introduced externally.

The pipeline — decided in session 1, never revised



Every arrow is a queue. Transcription runs in a separate process — a CPU-bound Whisper call in a thread would stall audio capture through the GIL.

Key architectural decisions

AUDIO TAP **WASAPI loopback (PyAudioWPatch)**

Only approach agnostic to meeting platform — Zoom, Teams, Meet, Slack all route through the OS mixer

VAD **Silero ONNX (vendored)**

The PyPI silero-vad package imports torch unconditionally — a second CUDA runtime on 4 GB VRAM is a real failure, not theoretical

ASR **faster-whisper · CTranslate2 · CUDA**

Local, free, language ID + punctuation in one pass; no cloud dependency

TRANSLATION **MyMemory REST API**

No billing setup; 5,000 words/day free. Google Cloud required billing even for the free tier.

SERVER / UI **FastAPI + vanilla HTML/CSS/JS**

No npm, no bundler, no build step between a change and a demo

Session segmentation strategy

Sessions were deliberately **segmented by role**: planning, design, implementation, and evaluation — each starting fresh with only the relevant docs loaded. Token

costs accumulate fast on a long project. A well-maintained doc set makes reloading context cheap enough that fresh sessions are always the right call.

72 decisions logged in DECISIONS.md — append-only, never rewritten. When a decision was superseded, a new entry said so. **PLAN.md**, **STATE.md**, **DECISIONS.md**, **INTERFACES.md** served as shared memory across all sessions and both LLM providers. Any tool, any machine — read those four files and the context is fully restored.

LEVELS 0-1 · AUDIO CAPTURE

The first real audio — a football interview in Spanish

The benchmark needed a sample that was real meeting-like speech, not studio-clean, in a non-English language. A **Lamine Yamal 60 Minutes interview in Spanish** was ideal: two speakers, natural pauses, occasional code-switching, the kind of audio Silero VAD is designed to handle.

⦿ The audio source: Lamine Yamal's extended 60 Minutes interview in Spanish — meeting-like, bilingual, natural pauses, real-world audio.

⦿ WASAPI loopback capture: 600 s · 18750 frames · 16 kHz mono int16 · streaming resample · peak -33.4 dBFS · drift -0.89 s.

⦿ loopback-20260819-141157.wav — 18.31 MB, 10 minutes, verified clean playback in the IDE audio player. Source: Realtek(R) Audio [Loopback] · int16 · 48000 Hz 2ch → 16000 Hz mono int16.

The resampling risk was closed by mutation test. A per-chunk resampler loses filter state at every chunk boundary: **-30.2 dB out-of-band energy** vs **-81.4 dB** streaming. One `soxr.ResampleStream`, never rebuilt. Injecting the bug into FrameFormatter fails exactly two tests and no others — the mutation test passes.

The VAD that was silently dead — and how we caught it

Silero VAD was integrated, the tests passed, `doctor` reported a clean run. Then a closer look: **0.0005 confidence on real recorded speech** where 1.0 was expected. The VAD was returning near-zero on every frame without raising an error.

D53 — The silent failure

The vendored file is Silero **v5**, which needs 64 samples of the previous frame prepended to the 512 new ones — a 576-wide call. All documentation said 512. The ONNX input dimension is dynamic, so the wrong call runs without error and returns garbage scores silently. The fix was also a discipline change: `doctor` now asserts `speech > 0.9` and `silence < 0.1` on known fixtures. A dead VAD fails the preflight.

- 🕒 `utts/` directory — every utterance as a listenable WAV, named by millisecond offset and close reason (silence or `max_length`). 160 utterances from 10 minutes of audio. VAD cost: 0.116–0.141 ms per 32 ms frame.

75 minutes of GPU work — and the DLL trap

The benchmark had to answer which model fits in 4 GB VRAM alongside the desktop compositor and browser, sustains RTF < 0.5 over ten minutes, and predicts p95 word age below 7 seconds. Seven configurations, ten wall-clock minutes each.

- Windsurf IDE — Claude Code running the full test suite (177 items collected) during the transcription worker implementation session. 93% passing at this checkpoint.

The DLL trap (D60) — five failed attempts

What happened

`doctor` passed. `ctranslate2.get_cuda_device_count()` returned 1. The model loaded in 3.6 s. The first transcription call died: *"RuntimeError: Library cublas64_12.dll is not found or cannot be loaded."*

Root cause: `os.add_dll_directory()` registers with Python's restricted DLL loader. CTranslate2's native code loads cuBLAS via `LoadLibrary` with a bare filename, which searches PATH — not the Python loader's directories. Fix: also prepend nvidia wheel bin dirs to `os.environ["PATH"]`. cuBLAS is not needed for device enumeration or weight loading — only for GEMM calls during inference.

Results — 7 configurations × 10 minutes

- BENCHMARK.md results table — transcribed directly from results.md. medium:int8_float16 selected (highlighted): RTF 0.474 sustained, VRAM 1849 MB, -10.6% throttling (getting faster over time, not slower).

MODEL	COMPUTE_TYPE	RTF (LAST MIN)	PEAK VRAM	GATE 1 (<0.5)	THROTTLING
small	float16	0.669	991 MB	FAIL	+0.2%
small	int8_float16	0.269	711 MB	PASS	-2.0%
medium	float16	1.502	2801 MB	FAIL	+8.7%
medium ✓	int8_float16	0.474	1849 MB	PASS	-10.6%
large-v3-turbo	float16	3.382	2369 MB	FAIL	+30.4%
large-v3-turbo	int8_float16	0.568	1353 MB	FAIL*	-6.3%
large-v3	int8_float16	0.694	3217 MB	FAIL	-7.4%

The GTX 1650 has no INT8 tensor cores. The assumption was that int8 would not beat float16 without them. **It did, dramatically: 2.3× for small, 3.2× for medium, 4.3× for turbo.** The int8 path is memory-bandwidth-bound, not compute-bound. Halving the weight size halves the bandwidth pressure regardless of tensor cores. float16 configs thermally throttled (+8 to +30%); int8 configs improved over time.

Both `small` and `medium` int8 configurations cleared the performance gates. The selection rule was "fastest that is accurate enough" — and "enough" requires listening. `small` garbled proper nouns, code-switched mid-sentence, and invented words. `medium` produced coherent, translatable Spanish. Decision: **medium:int8_float16.**

Wiring the full pipeline — tested without GPU or audio hardware

Levels 4 through 6 built the three remaining pipeline stages and wired them into a running web server with a WebSocket UI. Every stage was designed to be **testable without the hardware it depends on**.

The worker process pattern (D27)

Transcription runs in a **separate process**, not a thread. A CPU-bound Whisper call in a thread holds the GIL and stalls audio capture — a correctness requirement. The worker communicates over two `multiprocessing.Queue` instances. Load failure surfaces as `state="error"` in under two minutes, not a hang.

The async/sync bridge (D67) — the hardest part of Level 6

First attempt (failed)

```
result = await loop.run_in_executor(None, worker.out_queue.get)
# Works while running. But task.cancel() cannot be delivered
# because the thread is blocked on queue.get() with no timeout.
# asyncio.run() then hangs for 300 s waiting for the thread
pool.
```

Fix: three-part unblocking protocol

- `out_queue.get(timeout=0.5)` — thread exits within one window of `task.cancel()`
- `_stop()` puts `None` on the queue before cancelling — immediate unblock
- Bridge treats `None` as shutdown sentinel: `if result is None: break`

133 tests, zero GPU required

- `FakeWhisperModel` — returns a lazy generator; forgetting to drain it is detectable

- **FakeTranscribeWorker** — satisfies the exact same interface as the real worker
- **WavFileSource** — plays back the recorded WAV deterministically at any speed
- **DEMO_MODE=1** — switches the whole app to fake transcription for CI and review

Three hurdles before the first caption appeared

Hurdle 1 — Pipeline running, UI stuck at Idle

- ⦿ The first live UI — 127.0.0.1:8000. Pipeline started, status badge shows Idle, caption area completely empty. Start and Stop buttons present; nothing flowing.

Hurdle 2 — Audio captured, no subtitles

- ⦿ Status: Idle. YouTube playing alongside. Audio was captured but the pipeline state machine wasn't advancing past idle.
- ⦿ Status: Running (green badge), pipeline active — but still no caption cards. Audio reaching the pipeline; WebSocket not receiving translated output.

Hurdle 3 — Backpressure cascade

- ⦿ RTF 0.99, queue depth 4, utterances dropping continuously: "1 utterance(s) dropped (backpressure)." The controller was working correctly; the root cause was GPU resource contention.

Root cause: GPU resource contention

The GTX 1650 has no dedicated display engine. Chrome's hardware-accelerated video decode competes directly with CUDA on the same unified compute fabric, causing **WDDM preemption spikes of RTF 5–18×**. The backpressure controller (D28) was doing exactly what it was designed to do — the GPU was simply being taken away from it entirely. Fix: disable Chrome hardware acceleration (<chrome://settings/system>). One-time machine setup, now documented in WINDOWS.md.

FIRST SUCCESSFUL TRANSLATION

Spanish → English, live, in the browser

- ⦿ The first successful end-to-end run. ES (detected) → EN. Caption cards: translation primary (large, white) / source secondary (small, italic). Gap markers for silences > 1.2 s. RTF 0.81, word age 8.5 s, queue 0.

The UI showed all six defined states. Each caption card shows the translated text large and the original Spanish small beneath it — a user can distinguish a mishearing from a mistranslation without speaking the source language. Gap markers appear between utterances with significant preceding silence.

TRANSLATION API · MIGRATION

Google billed even for free tier — moved to MyMemory

The original plan was Google Cloud Translation — 500,000 characters/month free. In practice: Google Cloud requires a billing account before the free tier becomes accessible. That would also block anyone cloning the repo to review the system.

Alternatives considered

- **LibreTranslate** — open-source, self-hostable, free public API. Higher latency than Google's CDN.
- **MyMemory REST API** — selected. No key required; 5,000 words/day free, 10,000 with an email parameter.

The `GOOGLE_TRANSLATE_API_KEY` field in `.env` was repurposed: setting it to an email address passes it as MyMemory's `de=` parameter, doubling the daily limit. No billing, no account, no credential path. The character budget is **persisted across restarts** in `data_dir/usage/chars_YYYY_MM.json`.

11.3 minutes, all four gates — measured, not inferred

Level 7's job was to turn "it works" into "here are the numbers." A sustained 11.3-minute live run on the demo machine, browser open, JSONL log capturing every utterance's timestamps end-to-end.

5.9s

P95 WORD AGE

89%QUEUE DEPTH ≤ 1 **1375 MB**

PEAK VRAM

61°C

PEAK GPU TEMP

2

DROPS IN 11 MIN

0

CUDA OOM EVENTS

⦿ Sustained clean run — ES (detected) → EN. Header: RTF 1.01, age 5.5 s, q1, budget 2%. Bilingual caption cards, YouTube interview playing alongside. All gates passing.

Adaptive language detection (D69)

After the language is locked, Whisper re-runs without the language pin every 5 utterances. Two consecutive mismatches reset the lock — detecting a language switch within ~20 seconds. The `src = tgt` skip handles EN → EN pass-through without a translation round-trip.

All PRD Definition-of-Done items evidenced: system audio captured on Windows ✓ · source language auto-detected and overridable ✓ · target language selectable (15 languages) ✓ · captions never rewrite themselves ✓ · p95 word age 5.9 s < 7 s gate ✓ · output correctly punctuated ✓ · RTF < 0.5 sustained ✓ · UI needs no explanation ✓

From working to demonstrable — the floating window

The initial UI was functional but minimal. Once the pipeline was proven, a full redesign session overhauled the visual design and added the feature that makes the app genuinely usable in a real meeting: **a subtitle overlay that stays on top of the meeting window.**

- Windsurf — the redesign session. The old subtitle button (removed in the diff, shown in red) opened `/subtitle` in a new tab. User prompt: "the floating window is not good as I expected ... use the skill of `/ui-designer` to design and build the interface." Branch: `pip-subtitle-float-mode`.

Document Picture-in-Picture (D71)

The new approach uses the **Document Picture-in-Picture API** — a Chrome/Edge native capability that opens a browser tab as a floating window, always on top of all other windows, freely resizable and repositionable. The Subtitles button triggers it directly. No popup blocker, no second tab to manage, no Electron.

- The final product.** The subtitle overlay floats directly on top of the YouTube video, positioned and resized by the user. Translation primary (bold, large) / source secondary (small, italic). Frosted-glass background, slide-up animation on new captions. Always on top — works over any meeting client.

The redesigned main interface gained:

- **Mesh gradient background** with frosted-glass topbar and bottombar
- **Audio wave bars** — animated when speech is detected, flat when idle
- **State-driven badge glows** — six colours for six states (idle, loading, listening, running, degraded, error)
- **Age-based opacity** — recent captions full white, older captions fade gracefully
- **15 target languages** including Tamil, Malayalam, and Kannada
- **Docker image** — `docker compose up` runs a demo with no GPU or audio hardware required

Two LLMs, two IDEs, one codebase — without losing context

This project ran across **Linux and Windows**, used **Claude Code in Windsurf** and **Gemini in Antigravity IDE**, and spanned 14 sessions over several weeks. Keeping that coherent without re-explaining context in every session required deliberate tooling.

- CLAUDE.md — project rules for Windsurf and Claude Code sessions. Stack, platform constraints, hard rules (no PyTorch anywhere), and the one rule that matters most on the Windows machine.

- GEMINI.md — the identical counterpart for Antigravity and Gemini sessions. Explicit rule: these two files must never drift apart. Change one, change the other, say so in the commit.

Both files carry **identical project rules** — an explicit constraint so that switching LLMs mid-project doesn't introduce inconsistency. A new session with either tool reads the same rules, the same four docs, and is immediately context-aware.

- docs/journey/ — 12 session files, one per major phase. Each captures: the prompt used, what the LLM got right, where it needed pushing, the raw numbers, and what was learned. Built continuously from session 0.

Sessions were segregated by role: **planning, design, implementation, evaluation**. Each started fresh with only the relevant docs loaded. A planning session doesn't need to see test output. An implementation session doesn't need to re-read the benchmark methodology. Fresh context = lower token cost = longer effective runway per session.

What this project actually taught

On metrics

The most important reframing from session 0: **the metric that matters is word age, not RTF**. RTF measures whether the system is stable — can it keep up indefinitely. Word age measures whether the output is useful — does it arrive in time to follow a conversation. A configuration at RTF 0.49 with a 4-second utterance cap produces an 8-second word age. That passes the stability gate and fails the product requirement. Defining "minimal latency" precisely ($\text{word_age} = \text{silence} + \text{MT} + \text{D} \times (1 + \text{RTF})$) made the target measurable and model selection tractable.

On working with LLMs effectively

- **Tell it to push back.** "Do not agree with me" produces better output than "help me build X." The speaker-tagging correction and the Teams hard-stop both came from that single instruction.
- **Fresh sessions are cheaper than long sessions.** A well-maintained doc set makes reloading context cheap enough that fresh sessions are always the right call.
- **Inject the right scope.** A session told "you are implementing Level 4, read INTERFACES.md §4 only" produces more focused output than one handed the entire codebase.
- **Design for testability up front.** Every stage was designed with injection points. The LLM wrote those injection points because the spec asked for them explicitly — not as an afterthought.

On negative results

A documented negative result — the DLL trap with five failed attempts, the VAD returning 0.0005 on real speech, the backpressure cascade from GPU contention — is worth more than an untested assumption that happened to hold. That is exactly what the assignment is asking to see evidenced.

On documentation as infrastructure

The DECISIONS.md file (72 entries, append-only) and the session-based journey docs were not overhead — they were the mechanism that made 14 sessions across two machines, two IDEs, and two LLM providers coherent. Without them, every session would have rediscovered the DLL trap, the VAD width bug, and the Silero v5 context requirement independently.

Try it yourself

Full codebase on GitHub. Demo mode runs without GPU or audio hardware — reviewable on any machine.

```
# Demo mode — Docker, no GPU required
git clone https://github.com/victorsimon25/speech-
translator.git -b build
cd speech-translator/speech-translator
docker compose up --build
# open http://localhost:8000

# Full pipeline — Windows, GTX 1650 Ti or better
uv sync
python -m speech_translator.doctor
python -m speech_translator
```

→ [View build branch on GitHub](#)

★ [GitHub Repository](#)

5.9s

P95 WORD AGE

0.474

SUSTAINED RTF

1849 MB

PEAK VRAM

133

TESTS PASSING

Speech Translator · Build Journey · 2026 · victorsimon25